LeetCode 每日一题 (2026年05月16日)

今日主题：二分查找 (Binary Search) — 求平方根 & 搜索旋转数组

【第3天】二分查找 — 求平方根 & 搜索旋转数组

一、算法核心思想

二分查找的核心是每次将搜索区间减半，将时间复杂度从 O(n) 降到 O(log n)。

使用前提：搜索空间具有单调性（或可转化为单调性问题）。

二分模板：

- 左闭右闭 **[l, r]**：`while (l <= r)`，适合精确定位
- 左闭右开 **[l, r)**：`while (l < r)`，适合查找边界

二、经典题目：69. Sqrt(x)

题目描述

求非负整数 x 的算术平方根，向下取整。

```
输入：x = 8
输出：2
解释：sqrt(8) = 2.828..., 向下取整得 2
```

解法：二分查找

在 [0, x] 范围内二分查找最大的整数 mid 满足 $mid^2 \leq x$。

```
// 时间复杂度 O(log x)，空间复杂度 O(1)
int mySqrt(int x) {
    if (x <= 1) return x;

    int left = 0, right = x;
    int ans = 0;

    // 左闭右闭二分
    while (left <= right) {
        int mid = left + (right - left) / 2;  // 防止溢出

        // 用 long long 防止 mid*mid 溢出
        if ((long long)mid * mid <= x) {
            ans = mid;        // 记录当前可行解
            left = mid + 1;   // 尝试更大的值
        } else {
            right = mid - 1;  // mid 太大了，缩小右边界
        }
    }
    return ans;
}
```

牛顿迭代法（扩展思路）

```
// 牛顿法：x_{n+1} = (x_n + a/x_n) / 2
int mySqrtNewton(int x) {
    if (x == 0) return 0;
    double C = x, x0 = x;
    while (true) {
        double xi = 0.5 * (x0 + C / x0);
        if (fabs(x0 - xi) < 1e-7) break;
        x0 = xi;
    }
    return (int)x0;
}
```

三、经典题目：33. Search in Rotated Sorted Array

题目描述

一个原本升序的数组在某个点旋转后（如 [0,1,2,4,5,6,7] → [4,5,6,7,0,1,2]），在这个旋转数组中搜索 target。

```
输入: nums = [4,5,6,7,0,1,2], target = 0
输出: 4
```

解法：二分查找（利用局部有序性）

关键洞察：虽然数组整体不是有序的，但**mid** 一定将数组分成一个有序段和一个无序段。我们只在有序段中判断 target 是否在其中。

```
// 时间复杂度 O(log n)，空间复杂度 O(1)
int search(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) return mid;

        // 判断哪一半是有序的
        if (nums[left] <= nums[mid]) {
            // 左半段有序
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1;  // target 在左半段
            } else {
                left = mid + 1;   // target 在右半段
            }
        } else {
            // 右半段有序
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1;   // target 在右半段
            } else {
                right = mid - 1;  // target 在左半段
            }
        }
    }
    return -1;  // 未找到
}
```

解法对比

方法	时间复杂度	空间复杂度
二分查找	O(log n)	O(1)
先找旋转点再二分	O(log n)	O(1)
线性搜索	O(n)	O(1)

四、触类旁通

- **34.** 在排序数组中查找元素的第一个和最后一个位置 — 二分查找边界
- **162.** 寻找峰值 — 二分法爬坡
- **153.** 寻找旋转排序数组中的最小值 — 变种旋转数组